Softmax and log-sum-exp tricks

Richard Johansson

In many machine learning models including multinomial logistic regression and neural networks, we use the *softmax* function. The softmax is used to map a vector of scores $z_1, \ldots, z_N$ into probabilities $p_1, \ldots, p_N$. It is defined as

$$p_i = \frac{\exp z_i}{\sum_{k=1}^{N} \exp z_k}$$

where exp is the exponential function e^x. A naive Python implementation might look like this:

```
def bad_softmax(scores):
    expscores = np.exp(scores)
    return expscores / expscores.sum()
```

Note that this works with the whole array of scores. All the computations including `np.exp` are vector operations.

I referred to this implementation as "naive" because it is numerically unstable, due to the limited precision of floating-point numbers (which are normally limited to 64 bits). That is, we will see *numerical overflows*: for some values of z_i this code will crash because the numbers are too large when we call `np.exp`. Conversely, we may encounter *underflows*: if all z_i are large negative values, `np.exp` will return all zeros and we get a division by zero error. So how can we write the softmax function in a way that is numerically safe, without crashing?

The first trick is to work with the logarithms instead of the probabilities directly. In practice, computer implementations of probabilistic models almost always work with logarithms when this is possible. So we get

$$\log p_i = \log \frac{\exp z_i}{\sum_{k=1}^{N} \exp z_k} = z_i - \log \sum_{k=1}^{N} \exp z_k$$

The part after the minus sign is notable. This pattern is called "log-sum-exp" and occurs commonly in various types of machine learning and statistical models. But we still haven't made any progress dealing with the overflows and underflows in `np.exp`!

To get rid of these problems, we note that we can rewrite the log-sum-exp as follows:

$$\log \sum_{k=1}^{N} \exp z_k = \log \sum_{k=1}^{N} \exp(m) \exp(z_k - m) = m + \log \sum_{k=1}^{N} \exp(z_k - m)$$

This is true for any number m. So if we set m to be the maximal value among $z_1, \ldots, z_N$, then we will not have any overflows because the largest value used to call `np.exp` is 0. We will also avoid the underflow problem for the same reason: we know that at least one element returned by `np.exp` will be nonzero.

So to conclude: a numerically stable implementation could look like this:

```
def good_softmax(scores):
    m = np.max(scores)
    safe_scores = scores - m
    expscores = np.exp(safe_scores)
    return np.exp(safe_scores - np.log(expscores.sum()))
```

In practice, we typically don't have to do this work on our own, because we can rely on numerically stable implementations of softmax and log-sum-exp. These functions are available in many libraries, including mathematical libraries such as SciPy and machine learning libraries such as PyTorch.